Buildroot 快速使用手册

正点原子

正点原子公司名称 ：广州市星翼电子科技有限公司
原子哥在线教学平台：www.yuanzige.com
开源电子网 / 论坛 ：http://www.openedv.com/forum.php
正点原子淘宝店铺 ：https://openedv.taobao.com
正点原子官方网站 ：www.alientek.com
正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。

扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿：	正点原子 Linux 团队	正点原子 Linux 团队	2023.07.03

目录

目录

目录 4
免责声明 5
前言 6
第一章 简述 buildroot 与 SDK 7
1.1 SDK 与 buildroot 8
第二章 获取 buildroot 源码 9
2.1 开发板厂家提供 10
2.2 从 buildroot 官网下载 10
第三章 Ubuntu 开发 buildroot 注意事项 11
3.1 选择 Ubuntu 版本 11
3.2 新装 Ubuntu 选择源 11
第四章 搭建 buildroot 开发环境 16
4.1 解压 buildroot 源码 17
4.2 如何选择 buildroot 配置文件 18
4.3 编译 buildroot 常规软件安装 19
第五章 buildroot 编译常见问题 20
5.1 常见错误 1 忘记编译配置文件 21
5.2 常见错误 2 新增或者删除 package 编译 21
5.3 常见错误 3 如何重新/单独构建一个 package 21
5.4 常见问题 4 编译时下载失败 23
5.6 常见问题 5 编译源码失败 23
5.7 常见问题 6 编译 Qt/OpenCV 源码失败 23
附录 A 29

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

正点原子资料下载中心：http://www.openedv.com/docs/index.html

前言

本文旨在帮助初学者快速使用 buildroot，汇总了大量正点原子开发板用户反馈的问题，如果你是初学者，正好适合。本快速使用手册是基于标准 buildroot 源码讲解一些开发上的注意事项。

第一章 简述 buildroot 与 SDK

1.1 SDK 与 buildroot

SDK 中文解释为软件开发套件 (software development kit)。很多厂家开发板厂家如正点原子会提供开发套件给你，SDK 的类型有很多。在 Linux 我们常见的 SDK 类型有 Buildroot，Yocto 等。或者某些厂家提供了一个项目开发工具包给你，这也是一个 SDK。有些 SDK 是二次或者多次开发。如 RockChip（瑞芯微）提供的 Buildroot SDK，这个是 RockChip 基于 Buildroot 官方某个 Buildroot 版本开发的。他们会在某个 Buildroot 版本添加某款芯片添加软件支持，比如在 Buildroot 里添加 kernel，uboot 等 Linux 软件。为了方便构建他们的 SDK，他们会编写一些方便构建 SDK 的脚本。这个时候就与传统（Buildroot 原版）的构建指令有点不一样了，我们应当按照他们的 SDK 构建手册构建，尽管我们还能使用 Buildroot 里支持的指令。

在某些 SDK 里，不仅支持 Buildroot，还会支持 Yocto，Debian。这样的 SDK 集成了这么多开源的嵌入式 Linux 系统构建工具，SDK 必定会比较大。我们不必要担心，按他们的手册来使用这些 SDK 即可。

第二章 获取 buildroot 源码

介绍获取 Buildroot 源码的方法。

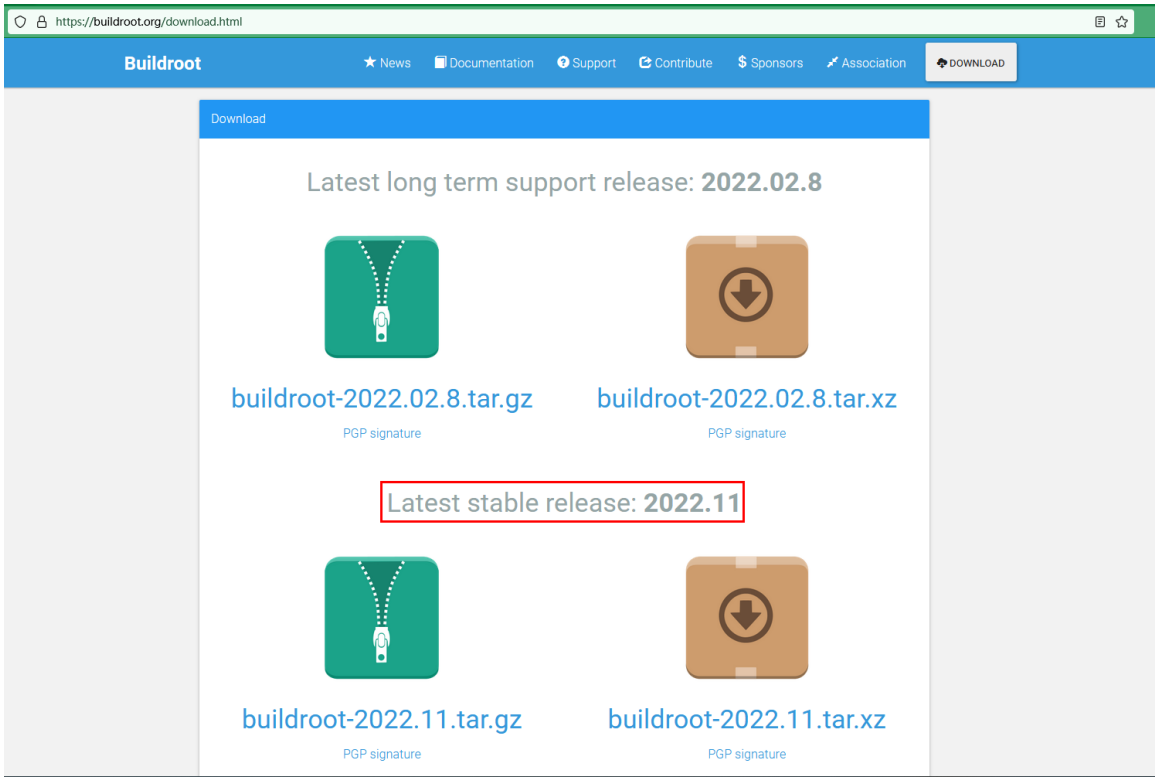
2.1 开发板厂家提供

一般开发板厂商会说明他们的 Linux 系统是 Buildroot 构建的，还是 Yocto 构建的，如果是 Buildroot 一般提供源码，如果源码是不开放的，那么可以寻问开发板厂家是基于哪个 Buildroot 官方版本开发的。我们就基于那个版本自己开发。

如果开发板厂家提供了源码，那么他们肯定会提供构建手册给你。我们按照他们的手册来构建即可。

2.2 从 buildroot 官网下载

正点原子很多款 Linux 开发平台都有 Linux 驱动开发指南，里面有讲述 Buildroot 基本使用。因为 Buildroot 每年都会更新，所以我们只能够以写这个使用手册为截止日期，每年的 2 月、5 月、8 月、11 月会发布一次 buildroot。下载 Buildroot 官网上最新的版本做测试。官网下载网址：https://buildroot.org/download.html。可以看到，当前最新的 buildroot 版本为 2022.11。好，我们点击下图的图标下载即可。这个就不浪费笔舌了，快速上手吧。



第三章 Ubuntu 开发 buildroot 注意事项

3.1 选择 Ubuntu 版本

我们一般使用开发板厂家推荐的 Ubuntu 版本开发。有些用户可能已经安装过 Ubuntu 虚拟机，为了省事就选择现有的虚拟进行开发，谁知道，这些用户的 Ubuntu 虚拟机环境已经被搞的乱七八糟。如果你是资深开发玩家，可以忽略 Ubuntu 环境的差异，因为资深开发玩家一般都会处理和解决开发环境的问题。如果你是初学者，那么你应该老老实实重新搭建一个开发板厂家推荐的 Ubuntu 版本开发。如果开发板厂家有提供搭建好的 Ubuntu 虚拟机，那就更好了。你也就省事了，可以直接用别人搭建好的 Ubuntu 虚拟机。如果你很享受学习开发过程，跟着搭建开发环境步骤，一路自己搭建也是一个不错的选择。

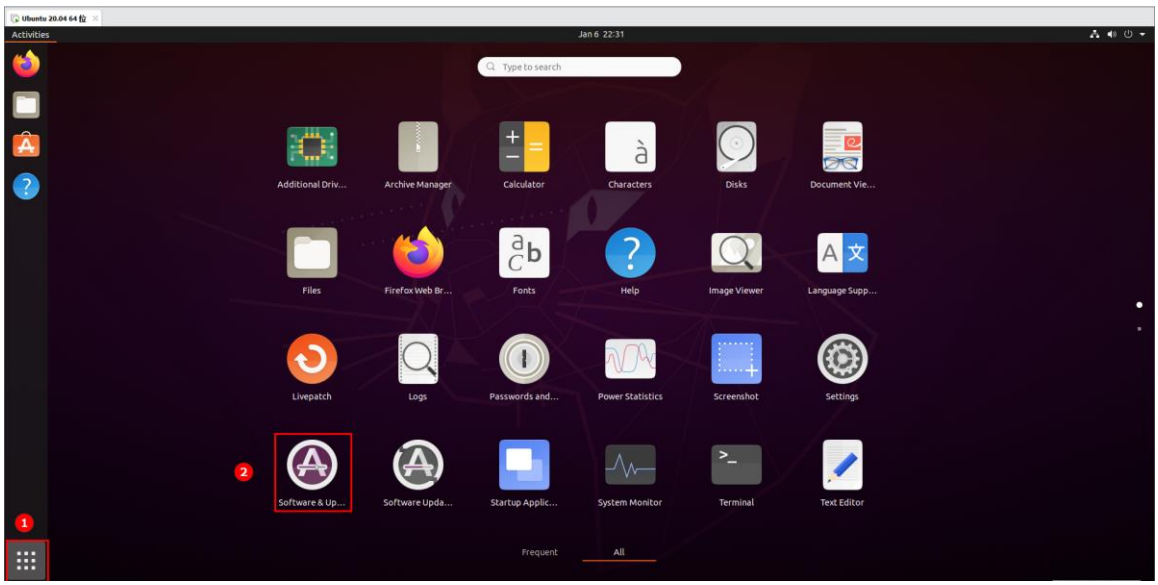
3.2 新装 Ubuntu 选择源

一般新装的 Ubuntu 第一步都要选择源，什么是源？源即软件源，也就是你要安装软件的服务器在哪。新装的 Ubuntu 默认的软件源在国外。所以我们会选择中国的服务器，服务器的远近这必定会影响你下载软件的速度。这好比如快递从国外送到你家和快递从中国杭州送到你家花费的时间长短是一个道理。

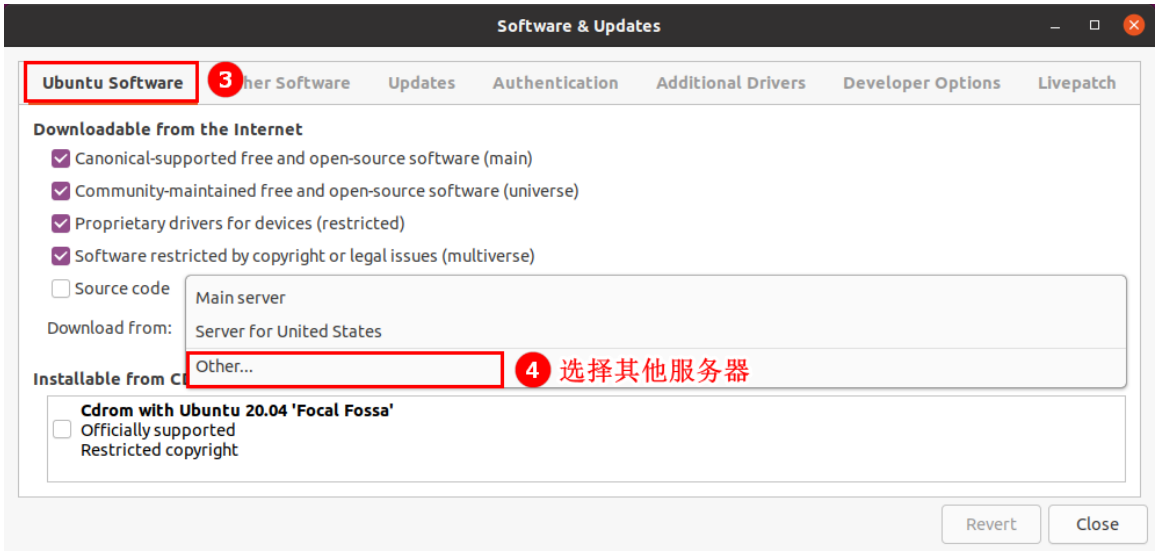
正点原子一般都是选择阿里云服务器。当然也有其他好用的中国服务器，大家可以尝试。如果你是正点原子的用户，个人建议你选择阿里云服务器，原子所有 Linux 开发环境都是用阿里云服务器测试过了的，你跟着来，用着也放心，不会因某个软件版本不一致导致编译失败，踩上了不必要的坑！

这里笔者也是一步一步的重新装一个 Ubuntu 来写文档，这是为了和大家的环境一样，重新安装了一个 Ubuntu 虚拟机。这里笔者使用的虚拟机版本为 Ubuntu20.04。如果你要安装其他版本的 Ubuntu 亦可！至少 Ubuntu16 以上。但是不同版本的 Ubuntu 需要安装的软件会不一样，因为有些软件已经早已经被废弃或者已经安装，但是这也不必要太担心，如果编译遇到错误，那么你将这个错误用百度搜索安装相应的软件再编译即可。但是笔者还是希望你使用与笔者相同 Ubuntu 版本来开发 buildroot。随着时间的推移 Ubuntu 版本会越来越高，文档还是适用的，不用担心，搭建环境的基本步骤是一样的。

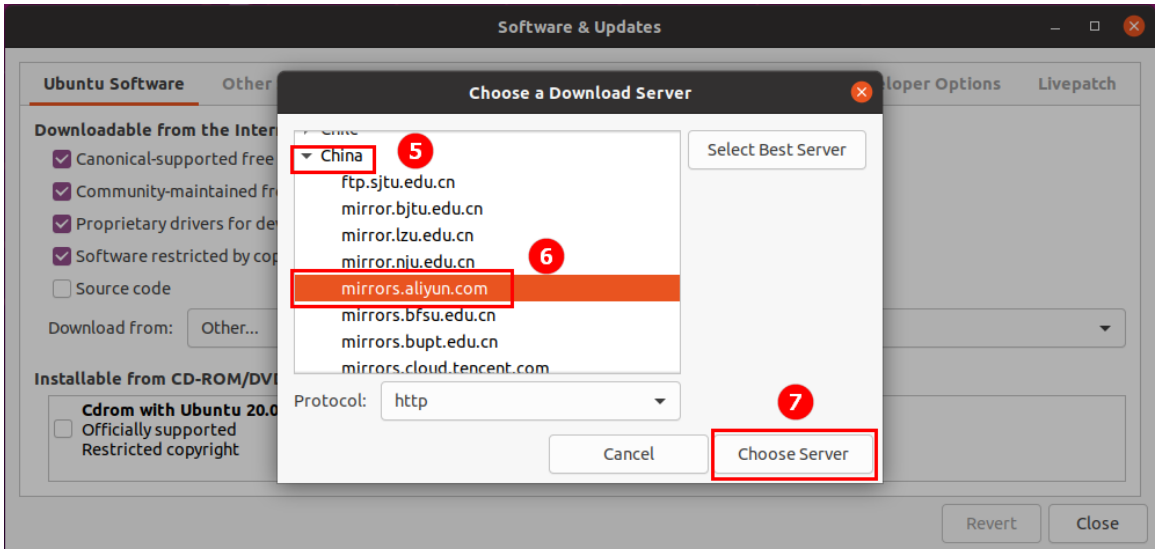
新装好的 Ubuntu20.04 想要安装软件，第一步需要更换软件源。如下图点击左下角，然后点击 SoftWare & Updates（软件与更新）。



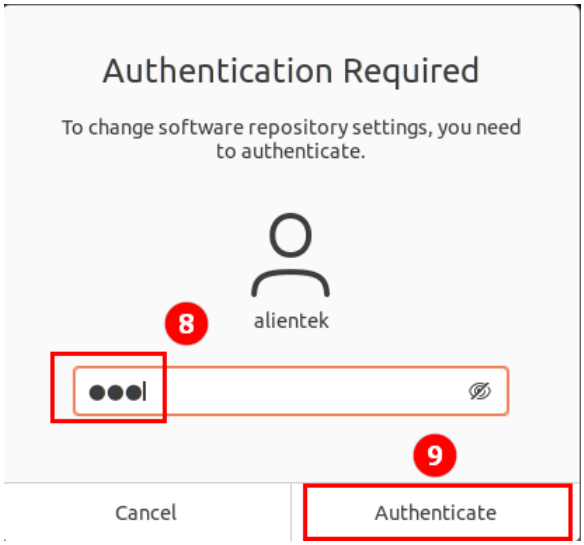
如下图，我们选择其他服务器，将软件源的服务器更改为中国的阿里云服务器。



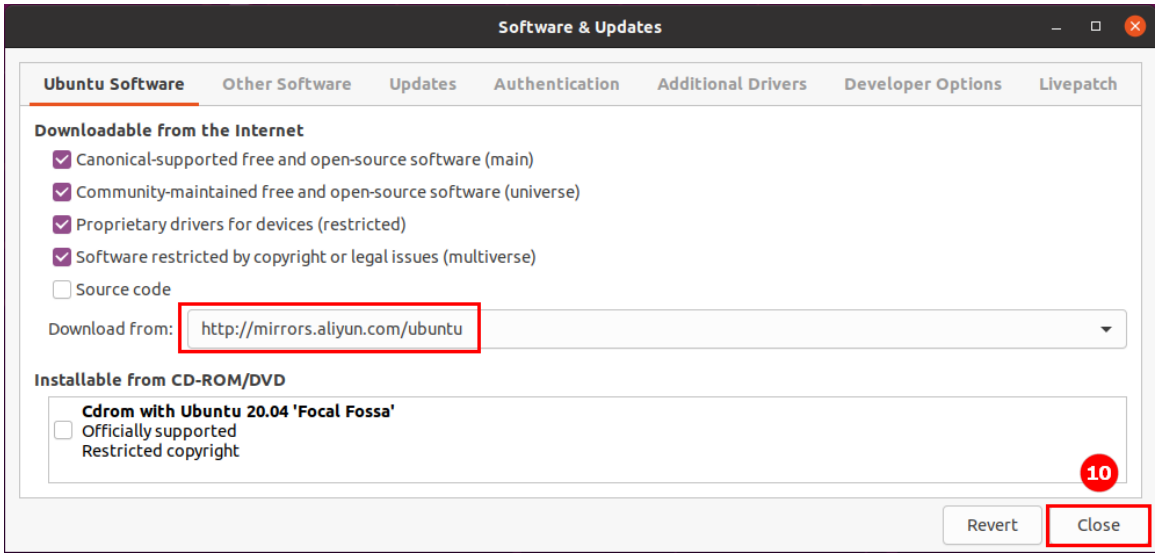
往前翻，找到 China 中国，然后再点击选择 mirrors.aliyun.com，然后再点击 Choose Server 选择服务器。



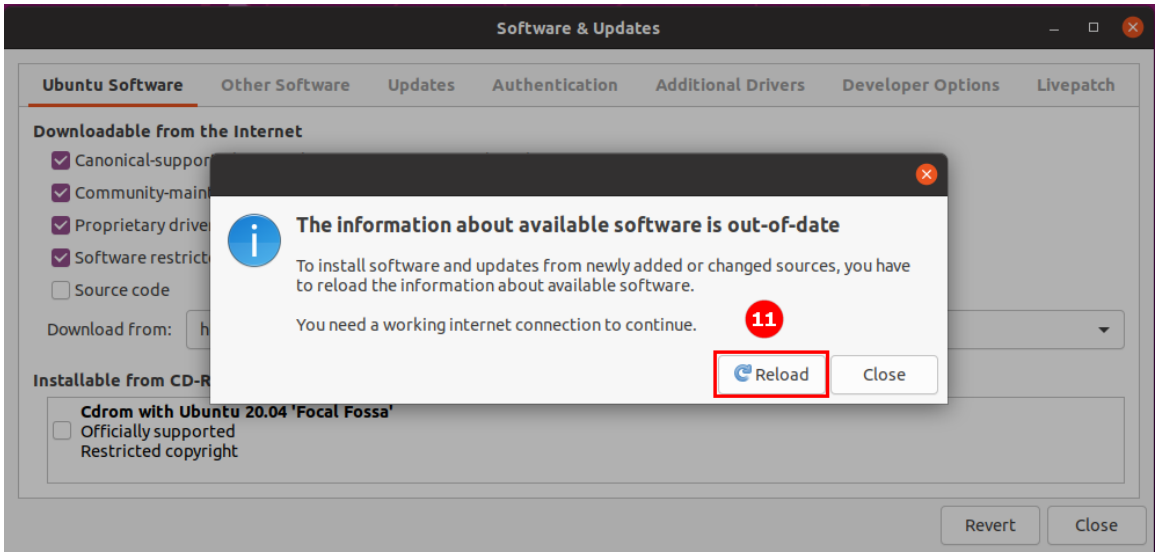
此时会弹出一个窗口输入的当前用户的密码确认，因为安装软件更新系统的东西都需要用户的权限才能继续。



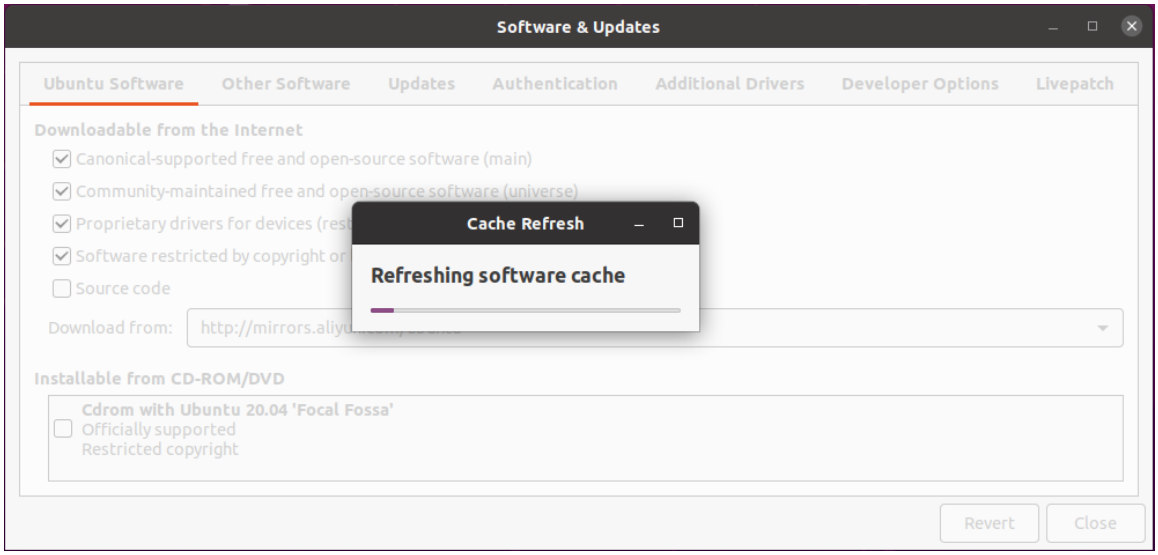
点击 Close 即可！



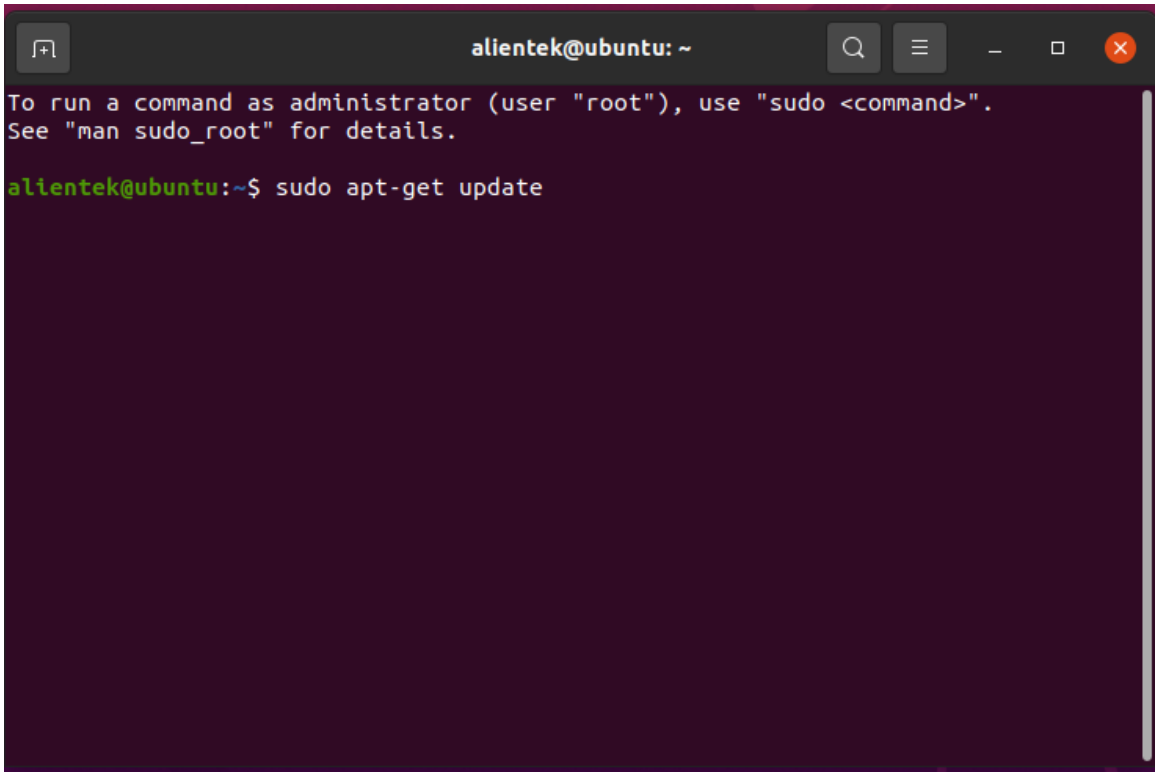
点击 Reload，重新加载。



点击后，Ubuntu 会刷新本地软件缓存，需要等待一段时间。直到它自动关闭即可。



在上一步我们更换了源，只是更换所指向的网址，那么我们安装的软件要重新刷新了，也就是更新本地的数据库，输入 sudo apt-get update 更新软件仓库里的软件。

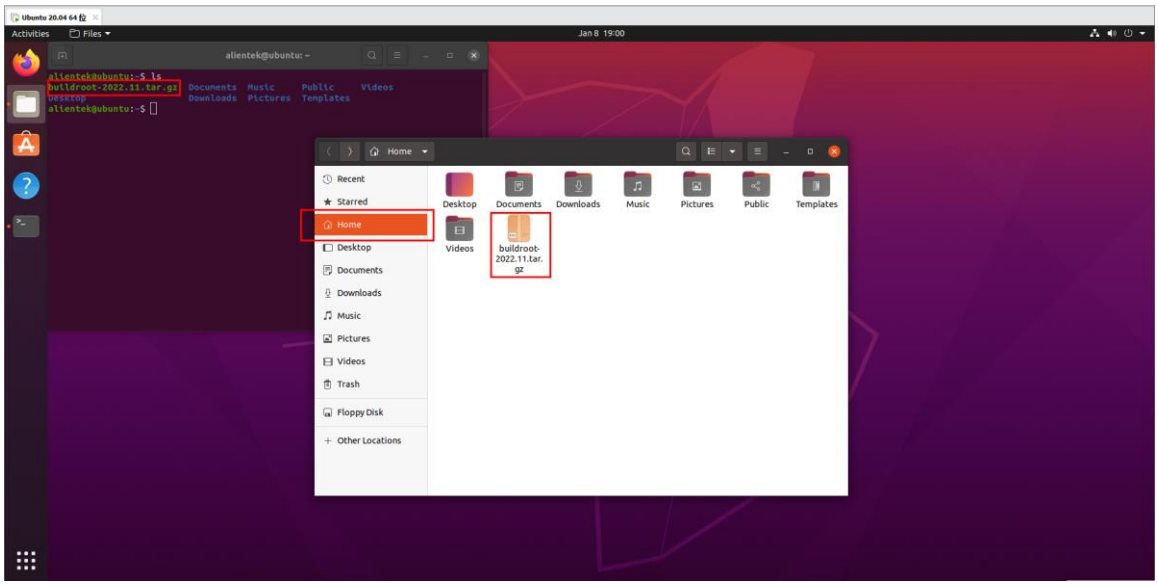


第四章 搭建 buildroot 开发环境

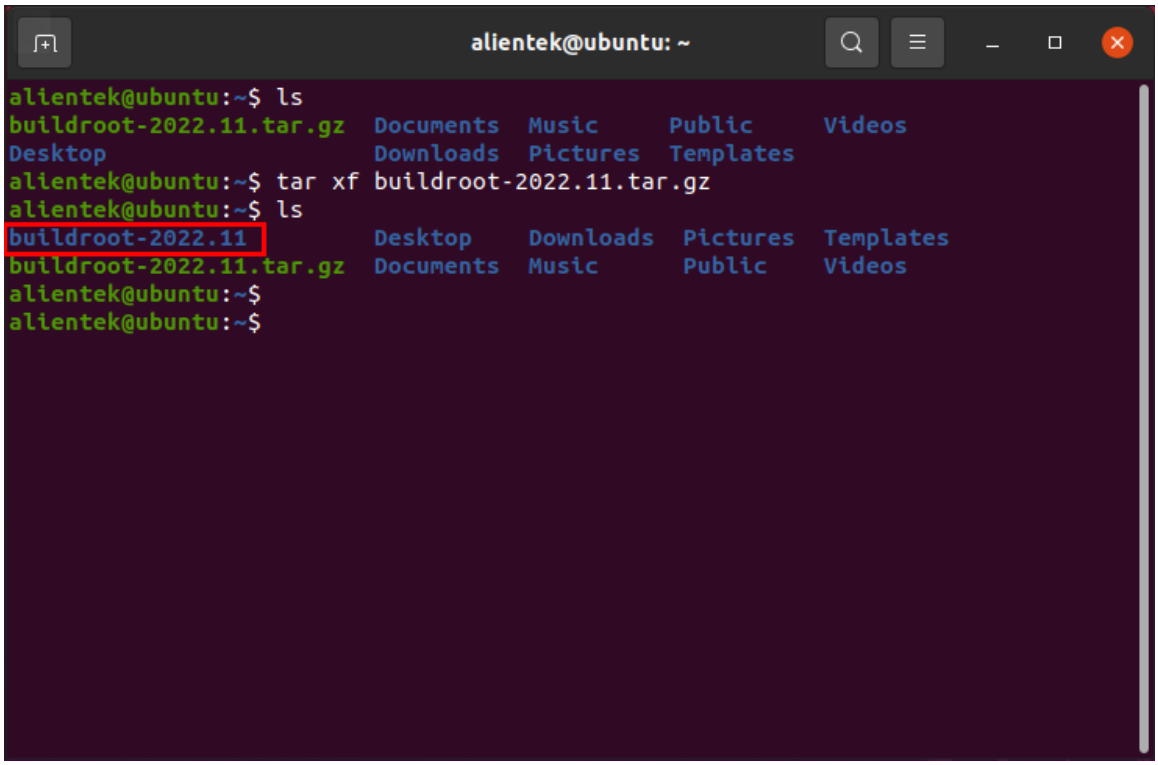
搭建开发环境，如果是开发板厂家提供了搭建环境的手册，那么我们就按照他们手册来搭建即可。这里我们肯定是统一不了 buildroot 的开发环境的。所以笔者只能基于官方 buildroot 来测试。在这里我们会讲解最基本的 buildroot 使用方法。掌握基本的，对于全局使用那就不难了！

4.1 解压 buildroot 源码

从 2.2 小节下载 buildroot 源码，笔者下载了 buildroot-2022.11.tar.gz 这个版本的源码。将它直接拷贝到 Ubuntu 虚拟机家目录。如果是初学者，不会用其它网络传输文件工具，我们可以直接从 Windows 将源码包拖到 Ubuntu 虚拟机家目录，如下图。



解压出来的 buildroot 源码如下图。



4.2 如何选择 buildroot 配置文件

一般情况下，由开发板厂家提供。虽然 buildroot 的 config/路径下有很多配置文件。我们可以使用指令查看。首先我们使用 cd 指令进入 buildroot-2022.11 文件夹下。

```
make list-defconfigs
```

此时会报错，新装的 ubuntu 连 make 工具都没有，所以我们先安装 make 工具。

```
sudo apt-get install make
```

```
alientek@ubuntu:~/buildroot-2022.11$ make list-defconfigs

Command 'make' not found, but can be installed with:

sudo apt install make        # version 4.2.1-1.2, or
sudo apt install make-guile  # version 4.2.1-1.2

alientek@ubuntu:~/buildroot-2022.11$ sudo apt-get install make
[sudo] password for alientek:
Reading package lists... Done
Building dependency tree
Reading state information... Done
Suggested packages:
  make-doc
The following NEW packages will be installed:
  make
```

再次输入 make list-defconfigs，如下。有很多板级支持配置 defconfig 文件。这些配置文件理论上是可以使用的。比如 imx6ullevk_defconfig 文件，理论上就是 imx6ull 的配置文件，可以编译给 imx6ull 开发板使用。但是需要注意，因为在这种官方支持的配置文件是针对某个内核版本的，你要修改这种配置文件要对应你的内核版本，否则文件系统会启动不能成功。这种修改还是要凭经验的。

```
alientek@ubuntu:~/buildroot-2022.11$ make list-defconfigs
/bin/bash: gcc: command not found
/bin/bash: line 0: [: : integer expression expected
Built-in configs:
  aarch64_efi_defconfig               - Build for aarch64_efi
  acmesystems_acqua_a5_256mb_defconfig - Build for acmesystems_acqua_a5_256mb
  acmesystems_acqua_a5_512mb_defconfig - Build for acmesystems_acqua_a5_512mb
  acmesystems_aria_g25_128mb_defconfig - Build for acmesystems_aria_g25_128mb
  acmesystems_aria_g25_256mb_defconfig - Build for acmesystems_aria_g25_256mb
  acmesystems_arietta_g25_128mb_defconfig - Build for acmesystems_arietta_g25_128mb
  acmesystems_arietta_g25_256mb_defconfig - Build for acmesystems_arietta_g25_256mb
  amarula_vyasa_rk3288_defconfig      - Build for amarula_vyasa_rk3288
  andes_ae350_45_defconfig            - Build for andes_ae350_45
  arcturus_ucls1012a_defconfig        - Build for arcturus_ucls1012a
  arcturus_ucp1020_defconfig          - Build for arcturus_ucp1020
  armadeus_apf27_defconfig            - Build for armadeus_apf27
  armadeus_apf28_defconfig            - Build for armadeus_apf28
  armadeus_apf51_defconfig            - Build for armadeus_apf51
  arm_foundationv8_defconfig          - Build for arm_foundationv8
  aspeed_ast2500evb_defconfig         - Build for aspeed_ast2500evb
  aspeed_ast2600evb_defconfig         - Build for aspeed_ast2600evb
  asus_tinker_rk3288_defconfig        - Build for asus_tinker_rk3288
  at91sam9260eknf_defconfig           - Build for at91sam9260eknf
  at91sam9g20dfc_defconfig            - Build for at91sam9g20dfc
  at91sam9g45m10ek_defconfig          - Build for at91sam9g45m10ek
  at91sam9rlek_defconfig              - Build for at91sam9rlek
  at91sam9x5ek_defconfig              - Build for at91sam9x5ek
  at91sam9x5ek_dev_defconfig          - Build for at91sam9x5ek_dev
  at91sam9x5ek_mmc_defconfig          - Build for at91sam9x5ek_mmc
  at91sam9x5ek_mmc_dev_defconfig      - Build for at91sam9x5ek_mmc_dev
  atmel_sama5d27_som1_ek_mmc_dev_defconfig - Build for atmel_sama5d27_som1_ek_mmc_dev
  atmel_sama5d2_xplained_mmc_defconfig - Build for atmel_sama5d2_xplained_mmc
  atmel_sama5d2_xplained_mmc_dev_defconfig - Build for atmel_sama5d2_xplained_mmc_dev
  atmel_sama5d3xek_defconfig          - Build for atmel_sama5d3xek
  atmel_sama5d3_xplained_defconfig    - Build for atmel_sama5d3_xplained
  atmel_sama5d3_xplained_dev_defconfig - Build for atmel_sama5d3_xplained_dev
  atmel_sama5d3_xplained_mmc_defconfig - Build for atmel_sama5d3_xplained_mmc
  atmel_sama5d3_xplained_mmc_dev_defconfig - Build for atmel_sama5d3_xplained_mmc_dev
  atmel_sama5d4_xplained_defconfig    - Build for atmel_sama5d4_xplained
  atmel_sama5d4_xplained_dev_defconfig - Build for atmel_sama5d4_xplained_dev
  atmel_sama5d4_xplained_mmc_defconfig - Build for atmel_sama5d4_xplained_mmc
  atmel_sama5d4_xplained_mmc_dev_defconfig - Build for atmel_sama5d4_xplained_mmc_dev
```

综上所述，我们还是使用开发板厂家提供的配置文件，毕竟有人为我们先测试了。

4.3 编译 buildroot 常规软件安装

输入下面的指令安装 buildroot 常规软件，没有这些工具，会编译报错。比如下面的 gcc 和 g++指令，在宿主机上，首先 buildroot 会编译后配置文件才能继续操作下去。

```
sudo apt-get install gcc g++

sudo apt-get install lsb-core lib32stdc++6 libncurses-dev
```

第五章 buildroot 编译常见问题及解决方法

详细技巧可以参考【正点原子】Buildroot 用户手册中文版(正点原子翻译)手册。buildroot 官方写的很全了大家可以查看。这里写得更口语化，总结了最常见的错误。

5.1 常见错误 1 忘记编译配置文件

首先我们需要配置 defconfig 文件，defconfig 文件就是要让你的 buidlroot 知道你要编译什么选项。很多用户经常是忘记配置 defconfig 文件，就直接 make 去编译了。编译出来的东西自己都不知道是什么东西。

```
make 配置文件
```

例如，如下。这个指令就是配置 imx6ull 的板级配置 defconfig 文件。我们先不管编译出来的东西能不能在我们的板子上用，执行下面的指令。

```
make imx6ullevk_defconfig
```

```
alientek@ubuntu:~/buildroot-2022.11$ make imx6ullevk_defconfig
mkdir -p /home/alientek/buildroot-2022.11/output/build/buildroot-config/lxdialog
PKG_CONFIG_PATH="" make CC="/usr/bin/gcc" HOSTCC="/usr/bin/gcc" \
    obj=/home/alientek/buildroot-2022.11/output/build/buildroot-config -C support/kconfig -f Makefile.br conf
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="<ncurses.h>" -DNCURSES_WIDECHAR=1 -DLOCALE  -I/home/alientek
/buildroot-2022.11/output/build/buildroot-config -DCONFIG_=\"\"  -MM *.c > /home/alientek/buildroot-2022.11/output/build/buil
droot-config/.depend 2>/dev/null || :
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="<ncurses.h>" -DNCURSES_WIDECHAR=1 -DLOCALE  -I/home/alientek
/buildroot-2022.11/output/build/buildroot-config -DCONFIG_=\"\"   -c conf.c -o /home/alientek/buildroot-2022.11/output/build/
buildroot-config/conf.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="<ncurses.h>" -DNCURSES_WIDECHAR=1 -DLOCALE  -I/home/alientek
/buildroot-2022.11/output/build/buildroot-config -DCONFIG_=\"\"  -I. -c /home/alientek/buildroot-2022.11/output/build/buildro
ot-config/zconf.tab.c -o /home/alientek/buildroot-2022.11/output/build/buildroot-config/zconf.tab.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="<ncurses.h>" -DNCURSES_WIDECHAR=1 -DLOCALE  -I/home/alientek
/buildroot-2022.11/output/build/buildroot-config -DCONFIG_=\"\"   /home/alientek/buildroot-2022.11/output/build/buildroot-con
fig/conf.o /home/alientek/buildroot-2022.11/output/build/buildroot-config/zconf.tab.o  -o /home/alientek/buildroot-2022.11/ou
tput/build/buildroot-config/conf
rm /home/alientek/buildroot-2022.11/output/build/buildroot-config/zconf.tab.c
#
# configuration written to /home/alientek/buildroot-2022.11/.config
#
alientek@ubuntu:~/buildroot-2022.11$
```

执行了上面的指令后，我们可以看到最后一句英文，buildroot 是将 imx6ullevk_defconfig 写进了当前目录下的.config 隐藏文件，这个配置文件作用是让 buildroot 知道它要编译什么，同时 make menuconfig 指令也会根据这个.config 来进行图形化选择菜单。

5.2 常见错误 2 新增或者删除 package 编译

很多用户在编译了成功 buildroot 后，发现少了某个功能，于是在图形菜单选上了 package 或者去除，然后编译。这个时候我们需要特别注意。如假的新增的 package 没有被其他的 package 依赖。此时我们就可以直接输入 make 再编译。如果你的 package1 有被其它的 package2 依赖，那么你就需对这个 package2 重新构建。

如果你更改了交叉编译工具链相关的包，那么建议你全部重新编译。因为大多数 package 包都是依赖工具链的。

5.3 常见错误 3 如何重新/单独构建一个 package

我们可以使用 make help 指令来查看帮助文档。如下，下为 make help 指令的部分截图。

```
alientek@ubuntu:~/buildroot-2022.11$ make help
Cleaning:
  clean                   - delete all files created by build
  distclean               - delete all non-source files (including .config)

Build:
  all                     - make world
  toolchain               - build toolchain
  sdk                     - build relocatable SDK

Configuration:
  menuconfig              - interactive curses-based configurator
  nconfig                 - interactive ncurses-based configurator
  xconfig                 - interactive Qt-based configurator
  gconfig                 - interactive GTK-based configurator
  oldconfig               - resolve any unresolved symbols in .config
  syncconfig              - Same as oldconfig, but quietly, additionally update deps
  olddefconfig            - Same as syncconfig but sets new symbols to their default value
  randconfig              - New config with random answer to all options
  defconfig               - New config with default answer to all options;
                               BR2_DEFCONFIG, if set on the command line, is used as input
  savedefconfig           - Save current config to BR2_DEFCONFIG (minimal config)
  update-defconfig        - Same as savedefconfig
  allyesconfig            - New config where all options are accepted with yes
  allnoconfig             - New config where all options are answered with no
  alldefconfig            - New config where all options are set to default
  randpackageconfig       - New config with random answer to package options
  allyespackageconfig     - New config where pkg options are accepted with yes
  allnopackageconfig      - New config where package options are answered with no

Package-specific:
  <pkg>                   - Build and install <pkg> and all its dependencies
  <pkg>-source            - Only download the source files for <pkg>
  <pkg>-extract           - Extract <pkg> sources
  <pkg>-patch             - Apply patches to <pkg>
  <pkg>-depends           - Build <pkg>'s dependencies
  <pkg>-configure         - Build <pkg> up to the configure step
  <pkg>-build             - Build <pkg> up to the build step
  <pkg>-show-info         - generate info about <pkg>, as a JSON blurb
  <pkg>-show-depends      - List packages on which <pkg> depends
  <pkg>-show-rdepends     - List packages which have <pkg> as a dependency
  <pkg>-show-recursive-depends
                          - Recursively list packages on which <pkg> depends
  <pkg>-show-recursive-rdepends
```

如下，如果我们需要对一个 package 重新构建或单独构建，那么我们可以执行下面的指令。清除 output/build 下的 package，包括源码及编译产物。注意这并不会清理安装产物

```
make <pkg>-dirclean
```

例如我们需要对 minicom 重新/单独编译

```
make <pkg>

make minicom-dirclean
```

重新编译需要清除再编译，若你没对源码修改只需要重新编译，那么你可以执行 make minicom-rebuild 指令重新编译。

重新编译/单独编译

```
make minicom
```

5.4 常见问题 4 编译时下载失败

在编译 buildroot 编译时，经常会遇到下载失败的问题，原因是网络问题，源码所在的服务器不响应你的请求了。此时我们需要停止编译。然后找到该 pakeage 的下载网址。一般编译时会打印出来的。如下，编译时下载 minicom 源码包，假设这个包没下载成功。那么我们根据它下载的网址，在 windows 使用网页浏览器/迅雷软件下载器将这个源码包下载好，然后将它拷贝到 buildroot 源码 dl 文件夹下。buildroot 下载的源码都会在这个 dl 文件夹下。dl 即是 download 的意思。

如 minicom 的源码包的路径。

https://salsa.debian.org/minicom-team/minicom/-/archive/2.8/minicom-2.8.tar.bz2。

下载好源码包后将其拷贝到 buildroot 源码的 dl 路径下。如果是 buildroot 官方源码，一般在 dl 路径下你还需要创建一个 minicom 文件夹，再将 minicom-2.8.tar.bz2 拷贝到这个 minicom 文件夹下。有些 buildroot 经过芯片厂商改过了，可能将源码包直接放到 dl 路径下即可，请根据 dl 路径下的源码包实际摆放执行相应的操作。

```
alientek@ubuntu:~/buildroot-2022.11/dl/minicom$ ls
minicom-2.8.tar.bz2
alientek@ubuntu:~/buildroot-2022.11/dl/minicom$
```

摆放好源码包之后，重新编译即可，就不会报源码下载失败的错误了。

5.6 常见问题 5 编译源码失败

一般情况下开发板厂商提供的 buildroot 根据他的配置方法来编译，一般不会出现源码编译失败问题。除非你修改了他的配置，或者新增了一些 package 来编译。buildrtoot 中的 package 并不是所有都可以编译通过。有些 package 要依赖编译器版本，有些要依赖主机环境（需要安装特定的软件工具包）。所以编译源码失败的情况会有很多种。一般如果是依赖编译器版本，所有依赖都会写在 package 下的 Config 文件里。

5.7 常见问题 6 因内存不足导致编译失败

这里我们忽略源码与编译器版本造成的编译失败。一般我们推荐使用开发板厂商（厂商测试过）及 buildroot 官方提供的 buildroot 里配置好的 Qt/OpenCV。如果是用户自己去修改编译器版本造成编译失败的情况不在讨论范围。

一般编译像 Qt/OpenCV 源码失败的原因是 Ubuntu 虚拟机（实体机）的内存太小。一般情况下 Ubuntu 虚拟机在默认安装时会默认分配 2G 的 SWAP 交换空间，也就是防止内存过小，使用磁盘空间当内作内存。但是远远不够编译 Qt/OpenCV，或许当你编译 Qt/OpenCV 失败时重启 Ubuntu 虚拟机再编译有可能会编译通过，因为重启释放某些内存/缓存了，现在就有足够的内存去编译 Qt/OpenCV 了。一般内存不够导致编译的错误如下，致命错误。

```
fatal error: Killed signal terminated program cc1plus
```

经过正点原子大量测试，Ubuunt 虚拟机至少要上 8G + 2G SWAP 空间才能编译过 Qt/OpenCV。这里的 Qt/OpenCV 指的是配置了很多 Qt/OpenCV 选项（正是因为某些选项需要下载大量的东西来构建造成内存不足），如果你只是编译 qtbase 或者 OpenCV 的核心组件，则无需增加内存也可编译过。

解决这样的问题有以下方法：

方法一：直接增加内存

一般是增加物理内存的方式是最快捷的了，增加物理内存（也就是加内存条），然后再给虚拟机分配 8G 或以上内存即可，如果是实体机，增加物理内存后直接编译即可。

方法二：删除编译项目

前面说了，正是因为我们配置了 Qt/OpenCV 的许多编译项造成内存不足的，我们可以尝试从最少的编译项开始编译，比如编译 Qt 则编译 qtbase，编译 OpenCV 那么我们编译 OpenCV core 核心组件即可。编译成功后，再按自己需要的项再增加编译即可。

方法三：增加交换空间

前面也说了，默认安装 Ubuntu 虚拟机默认有 2G 交换空间。假设现在你出现了内存不足的问题，现在你分配的虚拟机内存只有 4G，上面说过要编译 Qt 的全部项那么你需要 8G+2G 交换空间。实际上你的虚拟机内存只有 4G+2G。那么你还需要增加 4G 交换空间，理论这样才能编译 Qt 全部项。那么如何增加交换空间呢？(PS:这里建议使用固态硬盘安装 Ubuntu 虚拟机，否则磁盘读写速度不够，会导致磁盘占用率为 100%，有条件的就用固态，没有条件的机械也可以，笔者也是用了机械硬盘测试，成功编译)。

一般有两种方法。

第一种是安装 Ubuntu 虚拟机时，自己分配交换空间分区。

步骤如下：

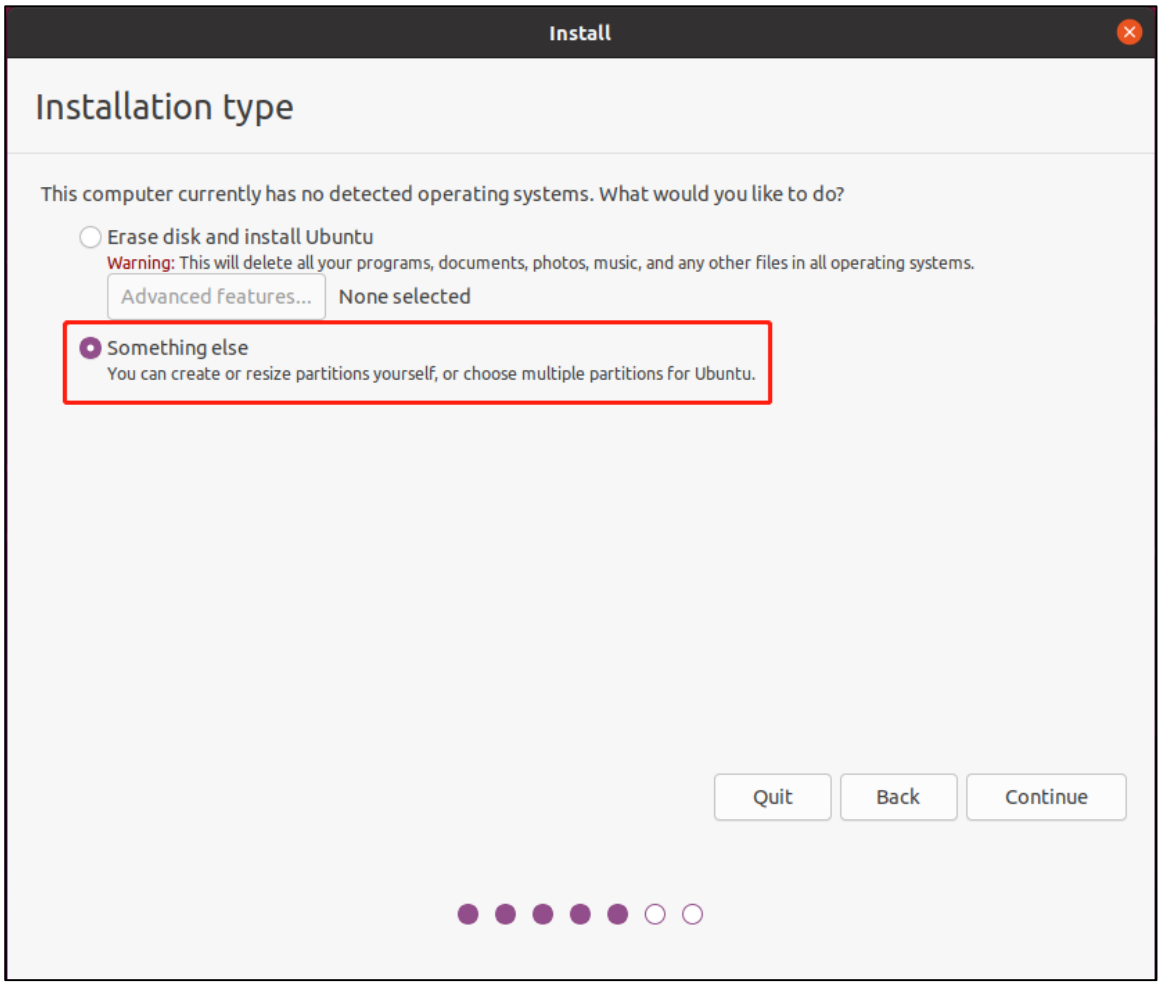
安装虚拟机时，请选择稍后安装操作系统。其他步骤按常规安装 Ubuntu 虚拟机即可。



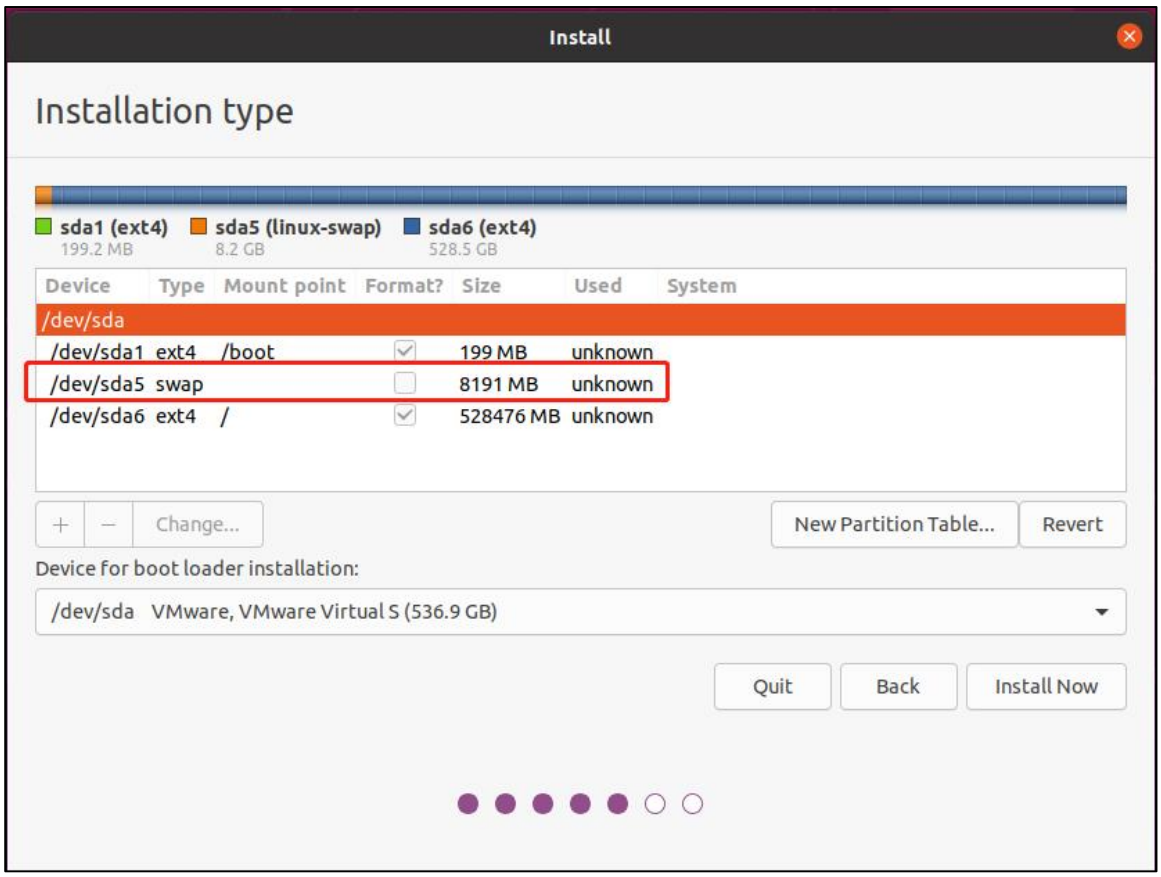
然后，在虚拟机设置里选择 CD/DVD，选择我们的 ISO 映像文件，也就是我们的 Ubuntu 文件。



进入安装界面时，请选择自定义分区。如下



如下，分区时建议将 swap 分配为物理内存的两倍，因为前面我们分配了 4G 物理内存，那么交换空间建议分配为 8G。那么你的 Ubunut 就可以使用“12G”内存了。然后点击默认安装即可。



安装完成的 Ubuntu 虚拟机在终端输入 free 指令可以看到。实际内存为 4G，交换空间约 8G。然后再重新搭建 buildroot 构建环境即可。

```
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alientek@alientek-virtual-machine:~$ free
              total        used        free      shared  buff/cache   available
Mem:        3982656      861976     2471304        2076      649376     2896648
Swap:       7999484           0     7999484
alientek@alientek-virtual-machine:~$
```

第二种是已经有 Ubuntu 虚拟机，配置好了环境，有些用户就不想重新安装 Ubuntu 了，那么我们如何为现有的 Ubuntu 虚拟机增加交换空间呢？

步骤如下：

同理如果是简易安装的 Ubuntu 那么在终端输入指令可以看到交换空间分区大小为 2GB。输入 free 指令。

```
alientek@ubuntu:~$ free
              total        used        free      shared  buff/cache   available
Mem:        3983200      996720     1882324        2168     1104156     2747240
Swap:       2097148           0     2097148
alientek@ubuntu:~$
```

我们要改变这个交换空间的大小，首先我们得关闭这个交换空间的大小再去改变。

```
sudo swapoff -a
```

输入关闭交换空间指令后再使用 free 指令查看，可以看到交换空间的大小变为 0 了。

```
alientek@ubuntu:~$ sudo swapoff -a
[sudo] password for alientek:
alientek@ubuntu:~$ free
              total        used        free      shared  buff/cache   available
Mem:        3983200      931368     1934680        2188     1117152     2809944
Swap:             0           0           0
alientek@ubuntu:~$
```

现在我们扩展交换空间大小，在 linux 下一切皆文件。所以交换空间可以当文件处理。这里我们创建了一个 8GB 的文件。

sudo dd if=/dev/zero of=/swapfile bs=1G count=8

```
alientek@ubuntu:~$ sudo dd if=/dev/zero of=/swapfile bs=1G count=8
8+0 records in
8+0 records out
8589934592 bytes (8.6 GB, 8.0 GiB) copied, 4.46635 s, 1.9 GB/s
alientek@ubuntu:~$
```

设置交换空间文件的权限。

sudo chmod 600 /swapfile

指定交换空间对应的设备文件

sudo mkswap /swapfile

```
alientek@ubuntu:~$ sudo mkswap /swapfile
Setting up swapspace version 1, size = 8 GiB (8589930496 bytes)
no label, UUID=da9f61a5-3168-4988-b681-8e5859dd786b
alientek@ubuntu:~$
```

启用交换分区

sudo swapon /swapfile

查看/etc/fstab 是否有/swapfile none swap sw 0 0，这样开机就会自动挂载这个交换空间。

cat /etc/fstab

```
alientek@ubuntu:~$ cat /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
# / was on /dev/sda5 during installation
UUID=a3ffe7d3-322e-4d5f-bd05-5683579a58f2 /               ext4    errors=remount-ro 0       1
# /boot/efi was on /dev/sda1 during installation
UUID=C4AF-3D16  /boot/efi       vfat    umask=0077      0       1
/swapfile                                 none            swap    sw              0       0
/dev/fd0        /media/floppy0  auto    rw,user,noauto,exec,utf8 0       0
alientek@ubuntu:~$
```

再次使用 free 指令查看交换空间的大小已经变为 8GB 了。

```
alientek@ubuntu:~$ free
              total        used        free      shared  buff/cache   available
Mem:        3983200      930480     1108228        2300     1944492     2782364
Swap:       8388604           0     8388604
alientek@ubuntu:~$
```

总结：如果是有条件的用户，建议增加物理内存，因为物理内存速度更快，增加交换空间这样的方式，磁盘当内存访问速度大大下降。如果你要一直开发，建议你配置一台好点的电脑，主板易升级内存，cpu 容易升级的主机。避免多次更换。

附录 A